Ai Assisted Coding

Ht.No:2303A52283

Batch:45

Assignment:11.3

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario:SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).

2. Implement the same functionality using a linked list for dynamic

memory allocation.

3. Implement the following operations in both approaches:

o Add a contact

o Search for a contact

o Delete a contact

4. Use GitHub Copilot to assist in generating search and delete methods.

5. Compare array vs. linked list approaches with respect to:

o Insertion efficiency

o Deletion efficiency

Prompt:

Generate a python code by using DSA. SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts. Implement using arrays and linked list and also include in arrays add a contact,search for a contact, delete for a contact.

Code:

```
class ArrayContactManager:
    def __init__(self):
```

```
self.contacts = []
    def add_contact(self, name, phone):
        self.contacts.append({'name': name.strip(), 'phone': phone.strip()})
    def search_contact(self, name):
        key = name.strip().lower()
        return [c for c in self.contacts if c['name'].lower() == key]
    def delete_contact(self, name):
        key = name.strip().lower()
        before = len(self.contacts)
        self.contacts = [c for c in self.contacts if c['name'].lower() != key]
        return before - len(self.contacts)
class Node:
    def __init__(self, name, phone):
        self.name = name.strip()
        self.phone = phone.strip()
        self.next = None
class LinkedListContactManager:
    def __init__(self):
        self.head = None
    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        if not self.head:
            self.head = new_node
            return
        curr = self.head
        while curr.next:
            curr = curr.next
        curr.next = new_node
    def search_contact(self, name):
```

```
        key = name.strip().lower()
        out = []
        curr = self.head
        while curr:
            if curr.name.lower() == key:
                out.append({'name': curr.name, 'phone': curr.phone})
            curr = curr.next
        return out
    def delete_contact(self, name):
        key = name.strip().lower()
        dummy = Node('', '')
        dummy.next = self.head
        prev, curr = dummy, self.head
        deleted = 0
        while curr:
            if curr.name.lower() == key:
                prev.next = curr.next
                deleted += 1
            else:
                prev = curr
            curr = curr.next
        self.head = dummy.next
        return deleted
if __name__ == '__main__':
    a = ArrayContactManager()
    a.add_contact('Alice', '111')
    a.add_contact('Bob', '222')
    print('Array search Bob:', a.search_contact('Bob'))
    print('Array delete Bob:', a.delete_contact('Bob'), 'left', a.contacts)
```

```
l = LinkedListContactManager()
l.add_contact('Alice', '111')
l.add_contact('Bob', '222')
print('Linked search Bob:', l.search_contact('Bob'))
print('Linked delete Bob:', l.delete_contact('Bob'))
print('Linked search Bob after:', l.search_contact('Bob'))
```

Output:

```
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>  & 'c:\Users\mouni\AppData\Local\Programs
n.exe' 'c:\Users\mouni\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\l
56442' '--' 'C:\Users\mouni\OneDrive\Desktop\Ai Ass labs\lab11_3.py'
Array search Bob: [{'name': 'Bob', 'phone': '222'}]
Array delete Bob: 1 left [{'name': 'Alice', 'phone': '111'}]
Linked search Bob: [{'name': 'Bob', 'phone': '222'}]
Linked delete Bob: 1
Linked search Bob after: []
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>
```

Explanation:

In this code we can code and output we can see that we have entered the contacts,array search bob, array delete boband so on.

TASK:02 Book Search System (Queues & Priority Queues)

Scenario:The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.

2. Extend the system to a Priority Queue, prioritizing faculty requests.

3. Use GitHub Copilot to assist in generating: enqueue() method , dequeue() method

4. Test the system with a mix of student and faculty requests.

Prompt:

Generate a python code for SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests. Also include Implement a Queue (FIFO) to manage book requests. Extend the system to a Priority Queue, prioritizing

faculty requests. enqueue() method , dequeue() method Test the system with a mix of student and faculty requests.In data structures.

Code:

```
import heapq

from collections import deque

from itertools import count

class Req:

    def __init__(self,id,type,title):

        self.id, self.type, self.title = id,type.lower(),title

    def __repr__(self): return f"{self.type[0].upper()}{self.id}:{self.title}"

class FIFO:

    def __init__(self): self.q = deque()

    def enqueue(self,r): self.q.append(r)

    def dequeue(self): return self.q.popleft() if self.q else None

class PQueue:

    def __init__(self): self.h, self.c = [], count()

    def enqueue(self,r):

        p=0 if r.type=="faculty" else 1

        heapq.heappush(self.h,(p,next(self.c),r))

    def dequeue(self): return heapq.heappop(self.h)[2] if self.h else None

reqs = [Req("1","student","DS"), Req("1","faculty","AI"),

        Req("2","student","Calc"), Req("2","faculty","ML"),
Req("3","student","Net")]

f, pq = FIFO(), PQueue()

for r in reqs:

    f.enqueue(r); pq.enqueue(r)

fifo_out = [f.dequeue() for _ in range(len(reqs))]

pri_out  = [pq.dequeue() for _ in range(len(reqs))]

print(f"{'FIFO':<20}{'PRIORITY':<20}")
```

```
print("-"*40)
for a,b in zip(fifo_out, pri_out):
    print(f"{str(a):<20}{str(b):<20}")
```

Output:

```
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>  c:; cd 'c:\Users\mouni\OneDrive\Desktop\Ai As
rs\mouni\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\mouni\.vscode\extension
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52758' '--' 'C:\Users\mouni\OneDrive\Des
ab11_3.py'
FIFO                 PRIORITY
----------------------------------------
S1:DS                F1:AI
F1:AI                F2:ML
S2:Calc              S1:DS
F2:ML                S2:Calc
S3:Net               S3:Net
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>
```

Explanation:

In this code we can see that which faculty is selecting which student and we can also see that which subject they are choosing.

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.

2. Provide the following operations:

o push(ticket)

o pop()

o peek()

3. Simulate at least five tickets being raised and resolved.

4. Use GitHub Copilot to suggest additional stack operations such as:

o Checking whether the stack is empty

o Checking whether the stack is full (if applicable)

Prompt:

Generate a python code for SR University's IT Help Desk receives technical support tickets from students and staff. While tickets are received sequentially, issue escalation follows a Last-In, First-Out (LIFO) approach. Implement a Stack to manage support tickets.Provide the following operations: push(ticket), pop(),peek().Simulate at least five tickets being raised and resolved. Use GitHub Copilot to suggest additional stack operations such as:Checking whether the stack is empty,Checking whether the stack is full (if applicable) By using data structures.

Code:

```
class Ticket:
    def __init__(self, i, u, iss):
        self.i = i
        self.u = u
        self.iss = iss
    def __repr__(self):
        return f"{self.i}:{self.u}:{self.iss}"
class Stack:
    def __init__(self, c=None):
        self.s = []
        self.c = c
    def push(self, t):
        if self.c is not None and len(self.s) >= self.c:
            raise OverflowError("Stack full")
        self.s.append(t)
    def pop(self):
        return self.s.pop() if self.s else None
    def peek(self):
        return self.s[-1] if self.s else None
    def is_empty(self):
        return not self.s
    def is_full(self):
```

```
        return self.c is not None and len(self.s) == self.c
st = Stack(10)
for x in [
    ("T1", "student", "Wi-Fi down"),
    ("T2", "staff", "Printer error"),
    ("T3", "student", "Password reset"),
    ("T4", "staff", "VPN timeout"),
    ("T5", "student", "Email sync")]:
    st.push(Ticket(*x))
print("peek:", st.peek())
print("empty:", st.is_empty(), "full:", st.is_full())
while not st.is_empty():
    print("resolved:", st.pop())
```

Output:

```
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>  c:; cd 'c:\Users\mouni\OneDrive\
rs\mouni\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\mouni\.vsc
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50138' '--' 'C:\Users\mouni
ab11_3.py'
peek: T5:student:Email sync
empty: False full: False
resolved: T5:student:Email sync
resolved: T4:staff:VPN timeout
resolved: T3:student:Password reset
resolved: T2:staff:Printer error
resolved: T1:student:Wi-Fi down
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>
```

Expalnation:

Stack class manages helpdesk tickets in LIFO order with `push`, `pop`, `peek`, `is_empty`, `is_full` and resolves newest tickets first.

Task 4: Hash Table Objective.To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

• Insert

• Search

• Delete

Starter Code

class HashTable:

pass

Prompt:

Generate a python code for : Hash Table Objective To implement a Hash Table and understand collision handling.

Task DescriptionUse AI to generate a hash table with: Insert , Search, Delete

Starter Code

class HashTable:

pass

Code:

```
class HashTable:
    def __init__(self, size=7):
        self.size = size
        self.buckets = [[] for _ in range(size)]
    def _idx(self, key):
        return hash(key) % self.size
    def insert(self, key, value):
        idx = self._idx(key)
        bucket = self.buckets[idx]
        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return
        bucket.append((key, value))
    def search(self, key):
        idx = self._idx(key)
```

```
        for k, v in self.buckets[idx]:
            if k == key:
                return v
        return None
    def delete(self, key):
        idx = self._idx(key)
        bucket = self.buckets[idx]
        for i, (k, _) in enumerate(bucket):
            if k == key:
                bucket.pop(i)
                return True
        return False
h = HashTable()
h.insert("a", 1)
h.insert("b", 2)
h.insert("a", 3 )
print(h.search("a"))
print(h.search("b"))
print(h.delete("b"))
print(h.search("b"))
```

Output:

```
PS C:\Users\mouni\OneDrive\Desktop\AI Ass labs>  c:; cd 'c:\Users\mouni\c
rs\mouni\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\mo
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62842' '--' 'C:\Use
ab11_3.py'
3
2
True
None
```

Explanation:

HashTable uses fixed-size buckets + chaining so `insert/search/delete` map keys to bucket by `hash(key)%size` and handle collisions in per-bucket list.

Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following

features:

• Student Attendance Tracking

• Event Registration System

• Library Book Borrowing

• Bus Scheduling System

• Cafeteria Order Queue

Student Tasks

1. Choose the most appropriate data structure for each feature.

2. Justify your choice in 2–3 sentences.

3. Implement one selected feature using AI-assisted code generation.

Prompt:

Generate a python code for Real-Time Application Challenge

Design a Campus Resource Management System with the following

features:Student Attendance Tracking,Event Registration System,Library Book Borrowing,Bus Scheduling System,Cafeteria Order Queue

Student Tasks:Choose the most appropriate data structure for each feature.

Justify your choice in 2–3 sentences.Implement one selected feature using AI-assisted code generation.

Code:

```
from collections import deque

from dataclasses import dataclass

@dataclass

class Order:

    id: int

    student: str

    items: list
```

```
    def __str__(self):
        return f"#{self.id} {self.student}: {', '.join(self.items)}"
class CafeteriaQueue:
    def __init__(self):
        self._queue = deque()
        self._next_id = 1
    def place_order(self, student_name, items):
        order = Order(self._next_id, student_name, items)
        self._next_id += 1
        self._queue.append(order)
        print("Order placed:", order)
        return order.id
    def serve_order(self):
        if not self._queue:
            print("No pending orders.")
            return None
        order = self._queue.popleft()
        print("Order served:", order)
        return order
    def pending_orders(self):
        return list(self._queue)
if __name__ == "__main__":
    q = CafeteriaQueue()
    q.place_order("Aisha", ["Sandwich", "Juice"])
    q.place_order("Ravi", ["Samosa", "Tea"])
    q.place_order("Mina", ["Pasta", "Water"])
    q.place_order("Anil", ["Burger", "Soda"])
    q.place_order("Sara", ["Salad", "Smoothie"])
    print("Pending:", q.pending_orders())
```

```
q.serve_order()
q.serve_order()
print("Pending after serving two:", q.pending_orders())
```

Output:

```
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>  c:; cd 'c:\Users\mouni\OneDrive\Desktop\Ai Ass labs'; & 'c:\Use
rs\mouni\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\mouni\.vscode\extensions\ms-python.debugp
y-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '58415' '--' 'C:\Users\mouni\OneDrive\Desktop\Ai Ass labs\l
ab11_3.py'
Order placed: #1 Aisha: Sandwich, Juice
Order placed: #2 Ravi: Samosa, Tea
Order placed: #3 Mina: Pasta, Water
Order placed: #4 Anil: Burger, Soda
Order placed: #5 Sara: Salad, Smoothie
Pending: [Order(id=1, student='Aisha', items=['Sandwich', 'Juice']), Order(id=2, student='Ravi', items=['Samosa'
, 'Tea']), Order(id=3, student='Mina', items=['Pasta', 'Water']), Order(id=4, student='Anil', items=['Burger', '
Soda']), Order(id=5, student='Sara', items=['Salad', 'Smoothie'])]
Order served: #1 Aisha: Sandwich, Juice
Order served: #2 Ravi: Samosa, Tea
Pending after serving two: [Order(id=3, student='Mina', items=['Pasta', 'Water']), Order(id=4, student='Anil', i
tems=['Burger', 'Soda']), Order(id=5, student='Sara', items=['Salad', 'Smoothie'])]
PS C:\Users\mouni\OneDrive\Desktop\Ai Ass labs>
```

Expalantion:

Cafeteria queue uses `deque` for FIFO order so orders are placed and served in real-world sequence.